

TJSC 2026 - Analista de Sistemas

Arquitetura de Sistemas e Integração

Aula 1 - Arquitetura de software, cliente-servidor, camadas, sistemas web e dispositivos móveis

Material autoral orientado ao edital retificado e ao padrão de cobrança da banca FGV

Campo	Descrição
Disciplina	Arquitetura de Sistemas e Integração - 10 questões na prova objetiva.
Aula	Aula 1 de 11.
Recorte da trilha	Arquitetura de software; arquitetura cliente-servidor; arquitetura em camadas; sistemas web; dispositivos móveis.
Não é o foco desta aula	MVC, DDD, microserviços, REST profundo, PDPJ, Spring, JPA, mensageria, Docker/Kubernetes e resiliência ficam para as próximas aulas.
Objetivo prático	Converter experiência com Angular, Java/Quarkus, APIs e integração corporativa em acerto de questão FGV.
Como usar: Leia a teoria com atenção nas tabelas de distinção. Depois resolva as 20 questões sem consultar o gabarito. Só então leia o gabarito resumido e os comentários completos.	

1. Protocolo aplicado antes da aula

A aula pedida é a Aula 1 da trilha de Arquitetura de Sistemas e Integração. Ela cobre o núcleo inicial do edital e prepara o vocabulário para as aulas seguintes. O edital vigente, retificado em 10/04/2026, lista a disciplina Arquitetura de Sistemas e Integração com os tópicos de arquitetura de software, arquitetura cliente-servidor, sistemas web e dispositivos móveis, MVC, DDD, microsserviços, integração, REST/JSON, webhooks, PDPJ, Spring, service discovery, API Gateway, persistência, mensageria, Docker, Kubernetes, Rancher e ambientes distribuídos. O recorte desta aula preserva os itens iniciais e não antecipa em profundidade os temas que terão aula própria.

Para calibrar a aula, foram usadas quatro referências de controle: o edital retificado do TJSC 2026; o Manual Mestre de Calibração FGV do projeto; provas e questões públicas da FGV em Tecnologia da Informação para observar formato, densidade de enunciado e alternativas; e referências primárias sobre HTTP, arquitetura web, camadas e arquitetura mobile. As questões são autorais e não reproduzem questão real.

2. Roteiro da aula

Bloco	Conteúdo	O que a FGV pode explorar
1	Conceito de arquitetura de software	Trocar arquitetura por ferramenta, linguagem, diagrama ou framework.
2	Arquitetura cliente-servidor	Confundir papel lógico com máquina física; dizer que cliente é sempre tela.
3	Sistemas web	Misturar browser, SPA, API, backend, servidor web, cache, DNS e TLS.
4	Dispositivos móveis	Ignorar offline, ciclo de vida, permissões, rede instável e sincronização.
5	Arquitetura em camadas	Confundir layer com tier, service com controller, repository com regra de negócio.
6	Trade-offs	Alternativas quase certas que exageram uma vantagem ou ocultam custo.
7	Questões autorais	20 itens variados, com gabarito separado e comentários completos.

3. Como a FGV tende a cobrar este bloco

A FGV, em Tecnologia da Informação, costuma cobrar mais do que definição isolada. É comum que o enunciado apresente um órgão público, uma equipe de TI, um sistema de atendimento, uma aplicação corporativa, um script, uma decisão arquitetural ou um pequeno fluxo. O candidato precisa identificar o conceito correto e eliminar alternativas que contêm termos verdadeiros em contexto errado.

No tema desta aula, o risco é o candidato experiente responder pela prática do dia a dia e não pela precisão conceitual. Em prova, arquitetura cliente-servidor, arquitetura em camadas, arquitetura em três tiers, sistema web, SPA, API, backend, servidor lógico e dispositivo móvel são conceitos próximos, mas não equivalentes.

Regra de prova: Quando a alternativa usa palavras como sempre, exclusivamente, necessariamente, basta, automaticamente ou elimina, desconfie. Em arquitetura, a FGV gosta de transformar uma boa prática contextual em regra absoluta.

4. Termos essenciais da aula

Termo	Expansão ou ideia	Leitura de prova
API	Application Programming Interface - Interface de Programação de Aplicações.	Contrato para que um software consuma funcionalidades ou dados de outro. Pode ser REST, SOAP, biblioteca, SDK etc.
Endpoint	Ponto de acesso de uma API.	Exemplo: GET /api/processos/{id}. Endpoint não é necessariamente a API inteira.
Request	Requisição.	Mensagem enviada pelo cliente ao servidor.
Response	Resposta.	Mensagem devolvida pelo servidor ao cliente.
Payload	Corpo útil da mensagem.	Dados enviados ou recebidos, frequentemente em JSON. Não inclui, necessariamente, cabeçalhos e

		metadados.
HTTP	Hypertext Transfer Protocol.	Protocolo de aplicação usado no modelo request/response da Web.
JSON	JavaScript Object Notation.	Formato textual de representação de dados; não é sinônimo de REST nem de API.
DNS	Domain Name System.	Resolve nomes como tjsc.jus.br para endereços IP. Não executa regra de negócio.
TLS	Transport Layer Security.	Camada criptográfica usada em HTTPS. Protege transporte, não valida automaticamente a lógica da aplicação.
Camada	Layer lógico.	Separação por responsabilidade no código: apresentação, aplicação, domínio, persistência.
Tier	Nível físico de implantação.	Separação por processo, servidor, nó, container ou ambiente.
Cluster	Conjunto de nós/instâncias trabalhando como unidade lógica.	Um servidor lógico pode ser composto por várias instâncias atrás de balanceador.

5. Arquitetura de software: o que é e o que não é

Arquitetura de software é o conjunto de decisões estruturais relevantes de um sistema: seus componentes principais, responsabilidades, dependências, formas de comunicação, distribuição, persistência, segurança e atributos de qualidade. Ela não se resume a desenhar caixas em um diagrama nem a escolher Spring, Quarkus, Angular ou Kubernetes.

Em prova, arquitetura é mais bem entendida como resposta a perguntas estruturais: quais partes existem? Como se comunicam? Onde ficam as regras de negócio? Como os dados são acessados? O que roda no cliente e o que roda no servidor? Como separar responsabilidades? Como a solução lida com manutenção, segurança, disponibilidade, desempenho e evolução?

Exemplo no seu contexto: um sistema interno com Angular no front-end, API Java/Quarkus no backend, autenticação corporativa, integração com serviços internos e Oracle/PostgreSQL como banco possui várias decisões arquiteturais. O fato de usar Angular ou Quarkus é tecnologia; a decisão de manter regra de negócio no backend, expor API para múltiplos clientes e separar Resource, Service e Repository é arquitetura.

5.1 Atributos de qualidade que aparecem por trás das alternativas

Atributo	Significado	Como pode aparecer na FGV
Manutenibilidade	Facilidade de modificar, corrigir e evoluir o sistema.	Separar responsabilidades reduz impacto de mudanças, mas não elimina disciplina técnica.
Testabilidade	Facilidade de testar partes isoladas.	Regra de negócio fora da UI e do Controller tende a facilitar teste.
Escalabilidade	Capacidade de lidar com aumento de carga.	Escalar servidor não exige, por si só, microsserviços; pode haver escala horizontal em monólito.
Disponibilidade	Capacidade de permanecer acessível.	Redundância, balanceamento e failover ajudam; serão aprofundados em aula própria.
Segurança	Proteção de dados e operações.	Centralizar regras sensíveis no servidor ajuda controle, auditoria e autorização.
Interoperabilidade	Capacidade de integrar com outros sistemas.	APIs, contratos e padrões de troca de dados são centrais.

5.2 Arquitetura, design e implementação

Nível	Pergunta	Exemplo	Pegadinha
Arquitetura	Qual é a estrutura principal e o trade-off?	Cliente Angular consome API Java; backend separado em camadas.	Dizer que arquitetura é apenas o diagrama.
Design	Como uma parte será organizada internamente?	Controller delega para Service; Repository abstrai acesso a dados.	Confundir padrão de classe com arquitetura do sistema inteiro.
Implementação	Como isso vira código/configuração?	@Path("/processos"), método GET, DTO, classe Service.	Confundir anotação de framework com conceito arquitetural.

6. Arquitetura cliente-servidor

A arquitetura cliente-servidor **descreve uma interação em que um cliente solicita um recurso ou serviço e um servidor processa a solicitação e devolve uma resposta**. Em sistemas web, esse padrão normalmente aparece com HTTP: o cliente envia uma request e o servidor responde com uma response. A MDN descreve HTTP como um protocolo cliente-servidor em que as mensagens do cliente são requests e as do servidor são responses.

O papel de cliente ou servidor é lógico e depende da interação. Um navegador é cliente quando chama uma API. Um aplicativo mobile é cliente quando chama o backend. Um backend A também pode ser cliente quando chama o backend B. O mesmo componente pode atuar como servidor em uma interação e como cliente em outra.

6.1 Elementos da arquitetura cliente-servidor

Elemento	Função	Exemplo
Cliente	Inicia a solicitação e consome o serviço.	Browser, Angular, app mobile, Postman, outro backend.
Servidor	Oferece recurso/serviço e responde às requisições.	API Java/Quarkus, serviço Spring Boot, servidor de autenticação.
Protocolo	Define regras de comunicação.	HTTP/HTTPS.
Contrato	Define como consumir o serviço.	Endpoint, método, parâmetros, payload, status codes.
Recurso	Objeto ou funcionalidade exposta.	/processos, /usuarios, /documentos.

6.2 Servidor lógico não é servidor físico

Uma alternativa pode dizer que o servidor responde às requisições. Isso não implica uma única máquina. O servidor pode ser um endpoint lógico exposto por várias instâncias atrás de um balanceador, por pods em Kubernetes, por containers ou por um serviço gerenciado. Para o cliente, existe uma URL ou endpoint. Por trás, pode haver múltiplos processos e nós.

Pegadinha: A presença de balanceador, cluster, container ou múltiplas instâncias não elimina o modelo cliente-servidor. Ela altera a implantação física, não o papel lógico de quem solicita e quem responde.

6.3 HTTP stateless e estado de sessão

HTTP é stateless: cada requisição, por si só, não carrega memória automática da requisição anterior no servidor. Isso não impede login, carrinho, autenticação, sessão ou token. Esses mecanismos são construídos sobre HTTP por meio de cookies, cabeçalhos, tokens, armazenamento no cliente e persistência no servidor.

Afirmção	Certo ou errado?	Por quê
HTTP é stateless.	Certo.	A requisição não mantém estado automaticamente entre chamadas.
Sistemas HTTP não podem ter login.	Errado.	Login usa mecanismos adicionais: cookie, token, sessão, banco, cache etc.
Stateless significa que o servidor nunca armazena dados.	Errado.	O servidor pode armazenar dados; o protocolo não preserva estado automaticamente entre requests.
JWT torna HTTP stateful.	Errado em regra.	Token pode carregar informação de autenticação, mas não muda a natureza básica do HTTP.

7. Sistemas web

Um sistema web é uma aplicação acessada por meio de tecnologias da Web, em geral usando navegador, HTTP/HTTPS, HTML, CSS, JavaScript e comunicação com servidores. Em sistemas modernos, é comum que a interface seja uma SPA - Single Page Application - em Angular, React ou Vue, consumindo APIs no backend.

A FGV pode cobrar sistemas web sem entrar em código. O foco costuma ser identificar responsabilidades: o navegador executa a interface e envia requisições; o backend aplica regras, controla segurança e acessa dados; o banco persiste informações; a rede transporta mensagens; DNS resolve nomes; TLS protege o canal de transporte em HTTPS.

7.1 Fluxo típico de um sistema web corporativo

```
Usuário -> Navegador/Angular -> HTTPS -> API Java/Quarkus -> Service -> Repository -> Banco de dados
      <- HTML/CSS/JS/JSON/status code <- resposta HTTP <- resultado processado
```

Esse fluxo pode ser descrito por ângulos diferentes. Como cliente-servidor, o navegador ou SPA é cliente e a API é servidor. Como camadas, há apresentação, aplicação, domínio e persistência. Como tiers, pode haver cliente no navegador, backend em servidor de aplicação e banco em servidor separado. Esses ângulos não são sinônimos.

7.2 SPA, MPA e renderização

Modelo	Ideia	Vantagem	Risco em prova
MPA - Multi Page Application	Servidor entrega páginas completas em múltiplas navegações.	Simplicidade conceitual, SEO e fluxo tradicional.	Dizer que é a única forma de sistema web.
SPA - Single Page Application	Uma aplicação JavaScript carrega no browser e atualiza partes da UI chamando APIs.	Boa experiência de usuário e separação front/back.	Achar que toda regra de negócio pode ficar no cliente.
SSR - Server-Side Rendering	HTML é gerado no servidor.	Pode melhorar primeira carga e indexação.	Confundir com arquitetura cliente-servidor inteira.
PWA - Progressive Web App	Aplicação web com recursos próximos de app, como cache e instalação.	Uso offline parcial e experiência mobile.	Achar que PWA é app nativo ou que dispensa backend.

7.3 Termos web que costumam aparecer em enunciado

Termo	Definição objetiva	Erro comum
URL/URI	Identificador/endereço de recurso.	Tratar URL como regra de negócio.
Cabeçalho HTTP	Metadados da mensagem.	Confundir header com payload.
Status code	Código que indica resultado da requisição.	Achar que 200 sempre significa regra de negócio concluída sem ressalva.
CORS	Mecanismo de controle de acesso entre origens no navegador.	Confundir com autenticação de usuário.
Cache	Armazenamento temporário para reuso.	Achar que cache substitui persistência transacional.
TLS/HTTPS	Proteção criptográfica do transporte.	Achar que resolve autorização e validação de negócio.

8. Dispositivos móveis

Dispositivos móveis entram no edital porque o cliente de um sistema não é apenas navegador desktop. Pode ser app Android, iOS, PWA, web responsivo, coletor, tablet institucional ou dispositivo usado por servidor/usuário externo. Arquiteturalmente, o ponto central é decidir o que fica no cliente móvel e o que permanece no servidor.

A documentação oficial de arquitetura Android recomenda, como princípio geral, separar pelo menos uma camada de UI e uma camada de dados, podendo haver uma camada de domínio opcional para encapsular lógica reutilizável ou complexa. A ideia de separação de responsabilidades é a mesma que a FGV pode cobrar em nível conceitual: UI não deve virar repositório de toda regra de negócio.

8.1 Diferenças arquiteturais entre web desktop e mobile

Aspecto	Sistema web comum	Sistema mobile
Rede	Tende a assumir conexão mais estável, embora isso não seja garantido.	Rede instável, troca entre Wi-Fi e 4G/5G, latência e desconexões.
Estado local	Pode usar storage/cookies/cache, mas frequentemente depende mais do servidor.	Pode precisar de cache local, fila local e sincronização posterior.
Ciclo de vida	A aba pode ser fechada, recarregada ou ficar em segundo plano.	App pode ser pausado, morto pelo sistema, retomar depois, receber push.
Permissões	Browser limita acesso a recursos do dispositivo.	App pode pedir câmera, localização, biometria, arquivos, notificações.
Distribuição	Atualização de frontend pode ser publicada no servidor/CDN.	App nativo passa por loja, MDM ou distribuição controlada; há versões antigas em campo.
Segurança	Depende de browser, HTTPS, cookies/tokens e backend.	Precisa lidar com armazenamento local seguro, jailbreak/root, engenharia reversa e expiração de tokens.

8.2 Offline-first e sincronização

Aplicações móveis podem precisar funcionar parcialmente offline. Isso não significa replicar o banco inteiro no aparelho nem abandonar o servidor. Significa projetar cache local, fila de operações, política de conflito e sincronização com o backend quando a conexão voltar.

Decisão	Quando ajuda	Risco
Cache local de leitura	Consulta frequente de dados pouco sensíveis ou que toleram desatualização.	Exibir dado vencido como se fosse confirmado.
Fila local de envio	Operações feitas em campo com rede instável.	Duplicidade, perda de idempotência e conflito.
Sincronização incremental	Grandes volumes ou atualização parcial.	Complexidade de versionamento e conflito.
Servidor como fonte de verdade	Regras sensíveis, auditoria, consistência e autorização.	Dependência de rede se não houver modo offline planejado.

9. Arquitetura em camadas

Arquitetura em camadas organiza o software por responsabilidades. A ideia é reduzir acoplamento, aumentar coesão, facilitar testes e localizar melhor mudanças. A documentação da Microsoft descreve que, conforme a aplicação cresce em complexidade, uma forma de gerenciá-la é separá-la por responsabilidades ou concerns, seguindo o princípio de separação de preocupações.

Em sistemas Java/Quarkus ou Spring, um desenho comum é dividir entrada web, aplicação, domínio, persistência e infraestrutura. A FGV pode chamar isso de arquitetura em camadas, separação de responsabilidades, divisão lógica ou organização por concerns.

9.1 Camadas típicas em backend corporativo

Camada	Responsabilidade	Exemplo Java/Quarkus	Erro comum
Apresentação/Interface	Receber requisições, validar formato básico, mapear entrada e saída.	Resource/Controller, DTO de entrada/saída.	Concentrar regra negocial complexa.
Aplicação	Orquestrar caso de uso, transação, autorização de fluxo e chamadas a domínio/infra.	ProcessoService, AprovarDocumentoUseCase.	Virar apenas repassador mecânico sem critério.
Domínio/Negócio	Representar regras centrais e invariantes do negócio.	Entidades, objetos de valor, políticas de validação.	Confundir entidade de domínio com tabela/DTO.
Persistência/Data Access	Abstrair acesso a dados.	Repository, Panache, JPA/Hibernate.	Colocar regra de autorização de negócio no SQL do repositório.
Infraestrutura	Implementar detalhes técnicos externos.	Cliente REST externo, mensageria, arquivos, cache.	Fazer a regra de negócio depender diretamente de framework.

9.2 Layer x tier: distinção que derruba

Layer é separação lógica no desenho do software. Tier é **separação física de implantação.** Um sistema pode ter várias camadas lógicas e rodar em um único processo. Também pode ter três tiers físicos e código internamente bagunçado. A FGV pode explorar exatamente essa diferença.

Conceito	Natureza	Exemplo	Frase correta
Layer	Lógica.	Controller -> Service -> Repository.	Organiza responsabilidades no código.
Tier	Física/deployment.	Browser, servidor de aplicação, servidor de banco.	Separa onde as partes rodam.
N-layer	Organização lógica em N camadas.	Apresentação, aplicação, domínio, persistência.	Pode estar em um monólito.
N-tier	Distribuição física em N níveis.	Front em CDN, API em cluster, banco gerenciado.	Pode usar rede entre tiers.

9.3 Monólito em camadas não é erro automático

Uma aplicação monolítica pode ser bem arquitetada em camadas. Monólito significa que o sistema é implantado como uma unidade principal, não necessariamente que o código é desorganizado. A FGV pode tentar sugerir que toda aplicação em camadas é microserviço ou que todo monólito é ruim. Ambas as generalizações são perigosas.

Para o TJSC, um sistema interno pode começar como monólito modular: um backend com módulos de processo, documentos, usuários e auditoria, todos no mesmo deploy, mas com separação clara de camadas. Isso pode ser uma decisão adequada quando a equipe, o domínio ou a necessidade de escala independente ainda não justificam microserviços.

10. Exemplo integrado: Angular + Quarkus + mobile

Cenário: o TJSC deseja disponibilizar consulta e acompanhamento de processos administrativos para usuários internos em uma interface web Angular e, futuramente, em um aplicativo móvel. A API backend será em Java/Quarkus, com acesso a banco relacional e autenticação corporativa.

```
Web Angular / App mobile
-> HTTPS /api/processos/{numero}
-> ProcessoResource
-> ConsultarProcessoUseCase ou ProcessoService
-> ProcessoRepository
-> Banco de dados
<- DTO em JSON + status HTTP
```

Ângulo de análise	Como classificar o cenário
Cliente-servidor	Angular e app mobile são clientes; API Quarkus é servidor na interação.
Sistemas web	Angular roda no navegador, consome API e renderiza a interface.
Dispositivos móveis	App mobile precisa lidar com rede instável, versões em campo, permissões e cache.
Camadas	Resource, Service/UseCase, Repository e banco separam responsabilidades lógicas.
Tiers	Pode haver browser/app, cluster backend e banco em ambientes físicos separados.

11. Trade-offs que a FGV pode transformar em alternativa

Decisão	Vantagem	Custo/risco	Alternativa maliciosa
Regra no servidor	Segurança, auditoria e consistência.	Mais chamadas de rede e dependência do backend.	Dizer que o cliente nunca deve validar nada.
Validação no cliente	Feedback rápido ao usuário.	Pode ser burlada; não substitui validação no servidor.	Dizer que validação no cliente garante segurança.
Camadas bem separadas	Manutenção, teste e troca de infraestrutura.	Pode gerar excesso de classes e repasse mecânico.	Dizer que mais camadas sempre significam melhor arquitetura.
SPA	Boa experiência e desacoplamento front/back.	SEO, bundle, segurança de token e complexidade no front.	Dizer que SPA dispensa backend.
Cache mobile/offline	Melhor disponibilidade percebida.	Conflitos, dados desatualizados e segurança local.	Dizer que cache é fonte de verdade.
Servidor lógico escalado	Disponibilidade e capacidade.	Exige balanceamento, estado externo e observabilidade.	Dizer que múltiplas instâncias deixam de ser cliente-servidor.

12. Pegadinhas de prova nesta aula

- Cliente não significa necessariamente tela. Um backend consumidor também pode ser cliente.
- Servidor não significa uma única máquina. Pode ser endpoint lógico com várias instâncias.
- HTTP ser stateless não impede sessão, login, cookie ou token.
- TLS/HTTPS protege o transporte, mas não substitui autenticação, autorização e regra de negócio.
- Camada é lógica; tier é físico. A banca pode trocar esses termos.
- API não é sinônimo de REST, JSON ou microserviço.
- Endpoint não é a aplicação inteira; é um ponto de acesso específico.
- Sistemas web podem ser MPA, SPA, SSR ou PWA; nenhum modelo é universalmente melhor.
- Dispositivo móvel impõe requisitos de rede, estado local, permissões, segurança local e versionamento.
- Validação no front melhora experiência, mas regra sensível precisa ser validada no servidor.
- Monólito em camadas pode ser arquitetura válida; microserviços não são pré-requisito para boa arquitetura.

13. Síntese de prova

Se o enunciado falar em...	Pense em...
cliente pede e servidor responde	Arquitetura cliente-servidor.
browser, Angular, HTML, JavaScript, API	Sistema web e cliente web.
app Android/iOS, rede instável, offline, permissões	Arquitetura para dispositivos móveis.

Controller, Service, Repository	Camadas lógicas.
browser separado de servidor de aplicação e banco	Tiers físicos.
payload JSON	Dados no corpo da mensagem; não necessariamente REST.
DNS, TLS, cache, CDN	Infraestrutura/protocolo de suporte, não regra de negócio central.
múltiplas instâncias atrás de balanceador	Servidor lógico escalado, ainda compatível com cliente-servidor.

14. Lista de questões autorais - padrão FGV

Instruções: resolva as 20 questões antes de consultar o gabarito. As questões foram elaboradas para treinar distinções finas entre arquitetura de software, cliente-servidor, camadas, sistemas web e dispositivos móveis. Não reproduzem questões reais da FGV.

Questão 1

Durante a revisão da arquitetura de um sistema do TJSC, a equipe descreveu que a solução terá uma aplicação Angular no navegador, uma API Java/Quarkus responsável por regras de negócio e um banco de dados relacional. Considerando o enfoque arquitetural, a descrição evidencia principalmente:

- (A) um padrão de criptografia de transporte, pois toda aplicação web depende de TLS para existir.
- (B) uma estrutura peer-to-peer, pois navegador e servidor executam código e, portanto, possuem as mesmas responsabilidades.
- (C) uma organização cliente-servidor combinada com separação de responsabilidades entre apresentação, aplicação e dados.
- (D) um modelo obrigatório de microsserviços, pois a presença de API elimina a possibilidade de monólito.
- (E) uma arquitetura de mensageria assíncrona, pois o navegador sempre se comunica com a API por fila.

Questão 2

Considere as afirmativas sobre arquitetura cliente-servidor. I. O cliente é o componente que inicia a solicitação em determinada interação. II. O servidor pode ser uma entidade lógica composta por várias instâncias físicas. III. Um backend que consome outro backend pode atuar como cliente nessa chamada. Está correto o que se afirma em:

- (A) I, apenas.
- (B) II, apenas.
- (C) I e III, apenas.
- (D) I, II e III.
- (E) II e III, apenas.

Questão 3

Em um sistema web, o navegador envia uma requisição HTTPS para uma API e recebe uma resposta com código de status e corpo em JSON. Nesse contexto, é correto afirmar que:

- (A) o payload corresponde ao endereço DNS usado para localizar o servidor.
- (B) a response é a mensagem retornada pelo servidor ao cliente, podendo conter status, cabeçalhos e corpo.
- (C) o endpoint é sempre o banco de dados acessado pela aplicação.
- (D) o código 200 garante, por si só, que todas as regras de negócio foram executadas corretamente.
- (E) o uso de JSON transforma automaticamente a arquitetura em event-driven.

Questão 4

Uma equipe afirma que seu sistema possui três camadas lógicas - apresentação, negócio e persistência - mas está implantado em uma única aplicação executada em um único servidor. A avaliação tecnicamente mais adequada é:

- (A) a descrição é contraditória, pois camadas lógicas exigem servidores físicos distintos.
- (B) a solução só pode ser classificada como mobile, pois toda aplicação em servidor único depende de cliente móvel.
- (C) a solução é necessariamente peer-to-peer, pois não há tier separado.
- (D) a solução deixa de ser arquitetura de software, pois não usa microsserviços.
- (E) a solução pode possuir arquitetura em camadas, pois layer é separação lógica e não necessariamente física.

Questão 5

No desenho de uma API Java, o Controller/Resource recebe a requisição HTTP, o Service orquestra o caso de uso e o Repository acessa o banco. A principal finalidade dessa estrutura é:

- (A) criptografar automaticamente o tráfego de rede entre cliente e servidor.
- (B) separar responsabilidades, reduzindo acoplamento e facilitando manutenção e testes.
- (C) substituir a necessidade de banco de dados relacional por armazenamento em memória.
- (D) permitir que a interface Angular execute diretamente consultas SQL complexas.
- (E) transformar qualquer monólito em arquitetura de microsserviços sem mudança de implantação.

Questão 6

Assinale a alternativa INCORRETA sobre sistemas web e dispositivos móveis.

- (A) Aplicações móveis devem, em regra, concentrar todas as regras sensíveis no próprio dispositivo, pois isso reduz a dependência do servidor e aumenta a segurança.
- (B) Sistemas web frequentemente utilizam HTTP/HTTPS para troca de mensagens entre cliente e servidor.
- (C) Aplicações móveis podem precisar lidar com conectividade intermitente, cache local e sincronização posterior.
- (D) Uma aplicação web SPA pode consumir uma API backend sem que isso, por si só, caracterize microsserviços.
- (E) A separação entre UI e dados é relevante tanto em aplicações web quanto em aplicações móveis.

Questão 7

Um aplicativo móvel permite ao usuário registrar uma solicitação mesmo sem conexão. A operação é gravada localmente e enviada ao backend quando a rede volta. O principal cuidado arquitetural nessa decisão é:

- (A) eliminar o servidor central, pois dados offline tornam a API desnecessária.
- (B) substituir HTTPS por DNS, já que o dispositivo móvel não realiza requisições seguras.
- (C) garantir que a View mobile execute diretamente as transações no banco central.
- (D) impedir qualquer armazenamento local, pois dispositivos móveis não podem armazenar dados.
- (E) tratar sincronização, conflitos, idempotência e segurança dos dados locais.

Questão 8

Observe o diagrama textual: Browser Angular -> HTTPS -> API -> Service -> Repository -> Banco. Considerando os conceitos da Aula 1, a leitura mais precisa é:

- (A) Repository é a camada de apresentação, pois recebe chamadas vindas da interface.
- (B) HTTPS é a camada de negócio, pois define as regras do caso de uso.
- (C) Browser Angular atua como cliente, API atua como servidor da interação e Service/Repository sugerem separação em camadas.
- (D) Banco atua como cliente, pois armazena a resposta final.
- (E) O diagrama prova, necessariamente, uma arquitetura em cinco microsserviços.

Questão 9

A respeito de HTTP, estado e arquitetura cliente-servidor, assinale a alternativa correta.

- (A) HTTP é stateful por definição, pois todo servidor web memoriza automaticamente cada usuário.
- (B) Como HTTP é stateless, sistemas web não podem implementar autenticação.
- (C) A existência de cookies ou tokens elimina a necessidade de servidor.
- (D) HTTP é stateless no protocolo, mas aplicações podem construir estado por sessões, cookies, tokens e persistência.
- (E) Stateless significa que o servidor nunca pode armazenar dados em banco.

Questão 10

Relacione os termos aos conceitos: 1. DNS. 2. TLS. 3. Payload. 4. Endpoint. () Proteção criptográfica do transporte em HTTPS. () Dados úteis enviados ou recebidos no corpo da mensagem. () Resolução de nomes para endereços. () Ponto de acesso específico de uma API. A sequência correta é:

- (A) 2 - 3 - 1 - 4.
- (B) 1 - 2 - 3 - 4.
- (C) 3 - 2 - 4 - 1.
- (D) 4 - 1 - 2 - 3.
- (E) 2 - 4 - 1 - 3.

Questão 11

Em uma arquitetura em camadas, uma decisão considerada adequada é:

- (A) permitir que o componente visual acesse diretamente o banco de produção para evitar latência.
- (B) colocar toda regra negocial no Controller, pois ele é o primeiro ponto da requisição.
- (C) manter regras centrais em serviços/domínio e acesso a dados em repositórios ou componentes de persistência.
- (D) tratar o banco como View, pois ele armazena dados que serão apresentados ao usuário.

(E) eliminar a camada de aplicação sempre que houver um endpoint HTTP.

Questão 12

Analise as afirmativas. I. Uma aplicação SPA pode ser cliente em uma arquitetura cliente-servidor. II. Uma API RESTful será aprofundada em aula própria, mas, nesta aula, já é possível reconhecê-la como uma forma comum de backend expor recursos a clientes. III. JSON é o protocolo de transporte que substitui HTTP em sistemas web modernos. Está correto o que se afirma em:

- (A) I, II e III.
- (B) I e II, apenas.
- (C) I e III, apenas.
- (D) II e III, apenas.
- (E) III, apenas.

Questão 13

Um tribunal possui um sistema web que precisa funcionar bem em desktop, tablet e celular, sem instalar aplicativo nativo. A solução adapta a interface ao tamanho da tela e usa recursos do navegador. O cenário descreve mais diretamente:

- (A) um message broker instalado no dispositivo.
- (B) uma arquitetura peer-to-peer entre usuários.
- (C) um banco de dados embarcado obrigatório.
- (D) uma substituição do backend por camada de apresentação.
- (E) um sistema web responsivo, possivelmente com características progressivas se usar recursos como cache e instalação.

Questão 14

Sobre layer e tier, assinale a opção correta.

- (A) Layer e tier são sinônimos exatos; ambos indicam apenas máquinas físicas.
- (B) Tier é sempre uma classe Java, enquanto layer é sempre um container Docker.
- (C) Layer indica separação lógica de responsabilidades; tier indica separação física ou de implantação.
- (D) Uma aplicação só possui layers quando usa Kubernetes.
- (E) Uma aplicação com três tiers nunca pode ter arquitetura em camadas.

Questão 15

Uma equipe propõe validar CPF, obrigatoriedade de campos e formato de data no Angular, mas repetir no backend as validações sensíveis e regras de negócio antes de persistir. A avaliação mais adequada é:

- (A) errada, pois qualquer validação no cliente torna a aplicação insegura por definição.
- (B) errada, pois validação no backend só é necessária em sistemas mobile.
- (C) indiferente, pois validações de formato e regras negociais são sempre equivalentes.
- (D) adequada: o cliente pode melhorar a experiência, mas o servidor deve proteger regras e persistência.
- (E) adequada apenas se o banco de dados for removido da arquitetura.

Questão 16

Em relação a sistemas web, assinale a alternativa que melhor representa uma característica de SPA - Single Page Application.

- (A) Carrega uma aplicação no navegador e atualiza a interface por chamadas a APIs ou recursos, sem recarregar páginas completas a cada interação.
- (B) Exige que todas as regras de negócio estejam no JavaScript do cliente.
- (C) É obrigatoriamente uma aplicação nativa instalada pela loja do sistema operacional.
- (D) Dispensa HTTP e utiliza somente DNS para comunicação.
- (E) Impede o uso de backend em Java ou Quarkus.

Questão 17

Uma aplicação de consulta processual possui três tiers físicos: cliente web, servidor de aplicação e servidor de banco. Internamente, o servidor de aplicação contém Controller, Service e Repository. Isso indica que:

- (A) a presença de três tiers impede separação em camadas lógicas.

- (B) tiers físicos e layers lógicos podem coexistir no mesmo sistema.
- (C) o banco de dados passa a ser a camada de apresentação.
- (D) o cliente web deve executar as consultas SQL para respeitar a separação física.
- (E) o sistema é necessariamente peer-to-peer.

Questão 18

Em um app móvel, parte dos dados de leitura é armazenada localmente para melhorar a experiência do usuário em rede instável. A alternativa que melhor expressa o trade-off é:

- (A) o cache local elimina a necessidade de autenticação no servidor.
- (B) dados em cache são sempre mais corretos do que os dados do backend.
- (C) a existência de cache transforma o app em servidor principal do sistema.
- (D) a sincronização deixa de ser necessária se o dado estiver no dispositivo.
- (E) o cache pode melhorar disponibilidade percebida, mas exige controle de validade, segurança e atualização.

Questão 19

Uma API está publicada em `api.tjsc.exemplo` e é atendida por cinco instâncias atrás de um balanceador de carga. Para o consumidor, há um único endereço lógico. A interpretação correta é:

- (A) a arquitetura cliente-servidor continua aplicável, pois o servidor pode ser lógico e composto por múltiplas instâncias.
- (B) o uso de balanceador transforma automaticamente a arquitetura em peer-to-peer.
- (C) a existência de cinco instâncias impede o uso de HTTP.
- (D) o cliente passa a ser o banco de dados, pois a API possui várias cópias.
- (E) o DNS substitui a camada de aplicação e executa as regras de negócio.

Questão 20

Na análise de uma arquitetura, a equipe decide separar apresentação, aplicação, domínio e infraestrutura. Contudo, o sistema ainda será implantado como uma única unidade, por simplicidade operacional. A conclusão mais adequada é:

- (A) a decisão é impossível, pois separação em camadas exige implantação independente de cada camada.
- (B) a solução deixa de ter arquitetura, pois arquitetura existe apenas em sistemas distribuídos.
- (C) a separação lógica só seria válida se houvesse aplicativo mobile.
- (D) é possível ter monólito modular em camadas, desde que as dependências e responsabilidades sejam respeitadas.
- (E) a presença de domínio e infraestrutura obriga o uso de DDD, que é o foco exclusivo desta aula.

15. Gabarito resumido

1	2	3	4	5
1-C	2-D	3-B	4-E	5-B
6-A	7-E	8-C	9-D	10-A
11-C	12-B	13-E	14-C	15-D
16-A	17-B	18-E	19-A	20-D

Gabarito em sequência: 1-C 2-D 3-B 4-E 5-B 6-A 7-E 8-C 9-D 10-A 11-C 12-B 13-E 14-C 15-D 16-A 17-B 18-E 19-A 20-D.

Observação editorial: o gabarito foi definido pela melhor alternativa de cada questão e não por quota matemática de letras.

16. Comentários completos

Questão 1 - Gabarito: C

Critério da correta: A alternativa C é a correta porque descreve o sistema por mais de um ângulo: há cliente web, servidor/API, regra de negócio e persistência, com separação de responsabilidades.

- (A) Tentadora por mencionar TLS, comum em aplicações web; errada porque TLS é proteção de transporte, não a arquitetura do sistema.
- (B) Tentadora porque navegador e servidor executam código; errada porque peer-to-peer exige maior simetria entre os nós, não apenas código nos dois lados.
- (C) Correta: combina cliente-servidor com separação lógica entre apresentação, aplicação e dados.
- (D) Tentadora por citar API; errada porque API não implica microserviços. Um monólito modular também pode expor API.
- (E) Tentadora por lembrar integração; errada porque o fluxo descrito é request/response síncrono, não mensageria por fila.

Questão 2 - Gabarito: D

Critério da correta: As três afirmativas estão corretas. Cliente e servidor são papéis na interação, e não rótulos fixos de produto ou máquina.

- (A) Errada porque considera apenas a afirmativa I e ignora que o servidor pode ser lógico e que backends também podem consumir serviços.
- (B) Errada porque considera apenas II, omitindo os papéis lógicos de cliente e a possibilidade de backend como cliente.
- (C) Tentadora por juntar I e III; errada porque a II também é verdadeira: servidor pode ser composto por várias instâncias.
- (D) Correta: I, II e III estão corretas.
- (E) Errada porque exclui a afirmativa I, que é a base do modelo cliente-servidor.

Questão 3 - Gabarito: B

Critério da correta: A response é a mensagem enviada pelo servidor ao cliente, normalmente com status, headers e, quando houver, corpo.

- (A) Tentadora por usar termo técnico; errada porque DNS resolve nomes, não é payload.
- (B) Correta: response é a resposta do servidor e pode conter status, cabeçalhos e corpo.
- (C) Tentadora por falar de acesso; errada porque endpoint é ponto de acesso da API, não o banco em si.
- (D) Tentadora porque 200 indica sucesso HTTP; errada porque sucesso HTTP não prova que toda regra de negócio foi semanticamente concluída sem ressalva.
- (E) Tentadora por associar JSON a integração; errada porque JSON é formato de dados, não arquitetura orientada a eventos.

Questão 4 - Gabarito: E

Critério da correta: É possível ter camadas lógicas mesmo com implantação em um único processo ou servidor. Essa é a distinção entre layer e tier.

- (A) Tentadora para quem confunde camadas com tiers; errada porque layer não exige servidor físico distinto.
- (B) Errada: servidor único não caracteriza sistema mobile.
- (C) Errada: ausência de tier separado não gera peer-to-peer.
- (D) Errada: arquitetura de software existe também em monólitos e sistemas simples.
- (E) Correta: camadas são separação lógica e podem coexistir com implantação única.

Questão 5 - Gabarito: B

Critério da correta: Controller/Resource, Service e Repository indicam separação de responsabilidades, melhorando manutenção, testabilidade e localização de mudanças.

- (A) Tentadora por ser requisito importante; errada porque criptografia de tráfego é papel de TLS/HTTPS, não da divisão Controller-Service-Repository.
- (B) Correta: a estrutura expressa separação de responsabilidades.
- (C) Errada: a arquitetura não substitui banco relacional por memória.
- (D) Errada: permitir SQL direto pelo Angular viola separação e segurança.

(E) Tentadora por associar camadas a modernização; errada porque camadas internas não transformam deploy em microsserviços.

Questão 6 - Gabarito: A

Critério da correta: A alternativa A é a incorreta. Regras sensíveis não devem ficar concentradas no dispositivo, pois cliente móvel é ambiente menos controlado.

(A) Incorreta: o app pode validar e operar localmente, mas regras críticas devem ser validadas no servidor.

(B) Correta: HTTP/HTTPS é base comum de comunicação web.

(C) Correta: mobile exige lidar com rede instável, cache e sincronização.

(D) Correta: SPA consumindo API não equivale automaticamente a microsserviços.

(E) Correta: separação de UI e dados é princípio aplicável tanto a web quanto a mobile.

Questão 7 - Gabarito: E

Critério da correta: Em operação offline, o ponto difícil é sincronizar depois com segurança, idempotência e tratamento de conflitos.

(A) Tentadora por parecer autonomia; errada porque offline não elimina o servidor central como fonte de verdade.

(B) Errada: DNS resolve nomes e não substitui HTTPS.

(C) Errada: app móvel não deve transacionar diretamente no banco central.

(D) Errada por ser absoluta; armazenamento local é possível e comum, com controles.

(E) Correta: sincronização, conflito, idempotência e segurança local são os cuidados arquiteturais.

Questão 8 - Gabarito: C

Critério da correta: O diagrama mostra cliente web, API como servidor da chamada e separação interna por camadas.

(A) Errada: Repository pertence à persistência/acesso a dados, não à apresentação.

(B) Errada: HTTPS é transporte seguro, não camada de negócio.

(C) Correta: o diagrama permite identificar cliente, servidor e camadas internas.

(D) Errada: banco armazena dados; isso não o torna cliente no fluxo.

(E) Tentadora por contar várias caixas; errada porque caixas/camadas não provam microsserviços.

Questão 9 - Gabarito: D

Critério da correta: HTTP é stateless no protocolo, mas aplicações constroem estado usando mecanismos adicionais.

(A) Errada: inverte o conceito; HTTP não é stateful por definição.

(B) Tentadora pela palavra stateless; errada porque autenticação é construída sobre HTTP.

(C) Errada: cookies e tokens ajudam a transportar/representar estado, mas não eliminam servidor.

(D) Correta: sessões, cookies, tokens e persistência podem construir estado de aplicação.

(E) Errada: stateless do protocolo não proíbe o servidor de armazenar dados em banco.

Questão 10 - Gabarito: A

Critério da correta: A sequência correta é TLS, payload, DNS e endpoint, isto é, 2 - 3 - 1 - 4.

(A) Correta: associa cada termo à função técnica adequada.

(B) Errada: começa tratando DNS como TLS.

(C) Errada: coloca payload como proteção de transporte e endpoint como resolução de nomes.

(D) Errada: transforma endpoint em TLS e DNS em payload.

(E) Tentadora por iniciar com TLS; errada ao trocar payload e endpoint.

Questão 11 - Gabarito: C

Critério da correta: A alternativa C coloca regra central e persistência em camadas adequadas.

(A) Tentadora por parecer reduzir latência; errada por acoplar UI ao banco e fragilizar segurança/auditoria.

(B) Tentadora porque Controller é a entrada; errada porque ele não deve virar classe que contém toda regra de negócio.

(C) Correta: regras centrais ficam em serviços/domínio; acesso a dados em repositórios/persistência.

(D) Errada: banco não é View; View é apresentação/representação ao usuário ou cliente.

(E) Errada: endpoint HTTP não elimina camada de aplicação.

Questão 12 - Gabarito: B

Critério da correta: I e II estão corretas; III está errada porque JSON não substitui HTTP.

- (A) Errada porque inclui III, falsa.
- (B) Correta: SPA pode ser cliente e API pode expor recursos a clientes.
- (C) Errada porque inclui III e exclui II.
- (D) Errada porque inclui III e exclui I.
- (E) Errada porque só III é falsa.

Questão 13 - Gabarito: E

Critério da correta: O cenário descreve sistema web responsivo; se houver cache, instalação e recursos progressivos, pode assumir características de PWA.

- (A) Errada: message broker é intermediário de mensagens, não solução de responsividade web.
- (B) Errada: usuários em browser não viram, por isso, pares simétricos peer-to-peer.
- (C) Tentadora em mobile; errada porque web responsivo não exige banco embarcado.
- (D) Errada: adaptar interface não substitui backend.
- (E) Correta: a descrição aponta para web responsivo e, possivelmente, PWA.

Questão 14 - Gabarito: C

Critério da correta: Layer é separação lógica; tier é separação física ou de implantação.

- (A) Errada: trata como sinônimos e reduz ambos a máquinas físicas.
- (B) Errada: mistura classe Java, layer e container sem relação conceitual.
- (C) Correta: é a distinção central da aula.
- (D) Errada: camadas independem de Kubernetes.
- (E) Errada: três tiers físicos podem coexistir com arquitetura em camadas.

Questão 15 - Gabarito: D

Critério da correta: Validar no cliente melhora experiência; validar no servidor protege regra, persistência e segurança.

- (A) Errada: validação no cliente não é proibida; ela apenas não basta para segurança.
- (B) Errada: validação no backend é necessária também em web.
- (C) Errada: formato de dado e regra de negócio não são equivalentes.
- (D) Correta: combina UX no cliente com proteção no servidor.
- (E) Errada: a presença de validações não implica remover banco de dados.

Questão 16 - Gabarito: A

Critério da correta: SPA carrega uma aplicação no browser e atualiza a interface por chamadas, frequentemente a APIs, sem recarregar toda a página em cada interação.

- (A) Correta: descreve a característica central de uma SPA.
- (B) Tentadora por ocorrer lógica no cliente; errada porque regra sensível não deve ficar toda no JavaScript.
- (C) Errada: descreve app nativo, não SPA.
- (D) Errada: DNS não substitui HTTP.
- (E) Errada: SPA pode consumir backend em Java/Quarkus sem problema.

Questão 17 - Gabarito: B

Critério da correta: Tiers físicos e camadas lógicas podem coexistir no mesmo sistema.

- (A) Errada: tier não impede layer.
- (B) Correta: cliente web, servidor de aplicação e banco são tiers; Controller/Service/Repository são layers internas.
- (C) Errada: banco não é apresentação.
- (D) Errada: cliente web não deve executar SQL direto para respeitar separação.
- (E) Errada: o cenário continua cliente-servidor, não peer-to-peer.

Questão 18 - Gabarito: E

Critério da correta: Cache local melhora a experiência, mas exige validade, segurança e atualização para não exibir dado incorreto ou vencido.

- (A) Errada: cache não elimina autenticação.
- (B) Errada: backend normalmente é a fonte de verdade.
- (C) Errada: cache não transforma app em servidor principal.
- (D) Errada: sincronização continua necessária para alinhar cliente e servidor.
- (E) Correta: expressa o trade-off arquitetural.

Questão 19 - Gabarito: A

Critério da correta: A arquitetura cliente-servidor permanece aplicável porque o servidor pode ser lógico e composto por múltiplas instâncias.

- (A) Correta: o cliente enxerga um endereço lógico e recebe resposta do serviço.
- (B) Errada: balanceador não cria peer-to-peer.
- (C) Errada: HTTP é amplamente usado com múltiplas instâncias.
- (D) Errada: banco não vira cliente por haver réplicas/instâncias da API.
- (E) Errada: DNS não executa regra de negócio.

Questão 20 - Gabarito: D

Critério da correta: É possível implantar como unidade única e ainda manter módulos/camadas bem definidos.

- (A) Errada: separação lógica não exige deploy independente.
- (B) Errada: sistemas não distribuídos também possuem arquitetura.
- (C) Errada: camadas não dependem de app mobile.
- (D) Correta: monólito modular em camadas é opção arquitetural válida.
- (E) Tentadora por mencionar domínio; errada porque DDD não é requisito automático e será tratado na Aula 2.

17. Checklist final da Aula 1

Item	Status
Arquitetura de software explicada como decisão estrutural, não como ferramenta.	Coberto.
Arquitetura cliente-servidor, request/response, HTTP e servidor lógico.	Coberto.
Sistemas web: browser, SPA, MPA, PWA, HTTP, DNS, TLS, payload e endpoint.	Coberto.
Dispositivos móveis: rede instável, cache local, permissões, offline e sincronização.	Coberto.
Arquitetura em camadas e distinção layer x tier.	Coberto.
20 questões autorais com formatos variados e comentários completos.	Coberto.
Sem flashcards e sem blocos escuros.	Coberto.

18. Referências consultadas

- TJSC/FGV - Edital retificado do Concurso Público TJSC 2026, Anexo de Conhecimentos Específicos, Arquitetura de Sistemas e Integração.
- Manual Mestre de Calibração FGV para Aulas e Questões - Projeto TJSC 2026.
- FGV Conhecimento - cadernos públicos de provas de Tecnologia da Informação, usados apenas para calibrar estilo, densidade e formato de alternativas.
- MDN Web Docs - Overview of HTTP. Disponível em: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview>
- MDN Web Docs - HTTP response status codes. Disponível em: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status>
- Microsoft Learn - Common web application architectures. Disponível em: <https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures>
- Microsoft Learn - Architectural principles. Disponível em: <https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/architectural-principles>
- Android Developers - Guide to app architecture. Disponível em: <https://developer.android.com/topic/architecture>
- Android Developers - Data layer. Disponível em: <https://developer.android.com/topic/architecture/data-layer>